

Предсказание возраста по фото лица

Отчет по вехе 6

Баранцов Данил, Костырин Николай

25 мая 2026 г.

1 Назначение и контракт

Сервис решает задачу регрессии возраста: по фотографии лица модель предсказывает возраст человека в годах (диапазон 0–116) и соответствующий возрастной бин (10 классов в разбивке датасета UTKFace).

Входной контракт. На вход подаётся изображение лица в форматах JPEG / PNG / WebP / BMP размером не более 5 МБ. Детекция и выравнивание лица не входят в скоуп сервиса: ответственность за подачу корректного фото лица возложена на вызывающую сторону, что согласуется с условиями обучающего датасета UTKFace.

Препроцессинг. Пайплайн полностью повторяет val-трансформ обучения: `Resize(256×256) → CenterCrop(224) → ToTensor → Normalize(ImageNet mean/std)`. Единственное определение трансформа вынесено в `src/inference/transforms.py` и используется как CLI-инструментами, так и сервисом — дублирования нет.

2 Базовая модель

По умолчанию сервис загружает дистиллированный ResNet18 (`notebooks/models/distilled_resnet18.pth`), полученный в ГП-5 методом knowledge distillation от учителя ResNet50. В таблице 1 приведены ключевые характеристики моделей, измеренные на val-split UTKFace (4 741 изображение).

Модель	RMSE	MAE	Acc@5	Acc@10	Размер
Teacher — ResNet50	8.01	5.60	59.5%	83.2%	90.0 МБ
Student без учителя — ResNet18	7.84	5.68	56.9%	82.5%	42.7 МБ
Дистиллированный — ResNet18	8.01	5.69	58.5%	82.1%	42.7 МБ
Pruned + fine-tuned — ResNet18	7.85	5.66	57.7%	83.0%	42.7 МБ

Таблица 1: Сравнение моделей по качеству и размеру (ГП-5)

Дистиллированный ResNet18 выбран моделью по умолчанию как наиболее компактный из полностью обученных (42.7 МБ, 11.7М параметров). Путь к чекпоинту и название архитектуры переопределяются через переменные окружения без изменения кода сервиса.

3 Общий слой инференса

Для исключения дублирования логики между CLI-скриптами и веб-сервисом инференс вынесен в отдельный пакет `src/inference/`. Пакет содержит три модуля:

- `transforms.py` — функция `build_val_transform(img_size=224)`: единый препроцессинг PIL-изображения, идентичный val-трансформу обучения.

- `weights.py` — `resolve_weights_path()`: разрешение пути к чекпоинту (аргумент → `MODEL_PATH` env → путь по умолчанию); `model_version(path)` — первые 8 символов SHA-256 файла весов, стабильный идентификатор чекпоинта.
- `predictor.py` — класс `AgePredictor`: загрузка `state_dict`, выбор устройства, методы `predict_bytes(raw_bytes)`, `predict_pil(image)`, `warmup()`; внутри держит `CachedAgePredictor` из `src.compression.caching` (двухслойный кэш из ГП-5).

Ядро инференса (`AgePredictor.predict_pil`):

```
def predict_pil(self, image: Image.Image) -> dict:
    before_hits = self._cached.pred_cache.hits
    age = self._cached.predict_single(image, self._transform)
    age = float(np.clip(age, 0, 116))
    was_cached = self._cached.pred_cache.hits > before_hits
    return {
        "age": round(age, 2),
        "age_bin": age_to_bin(age),
        "cached": was_cached,
        "model_version": self._version,
    }
```

4 Архитектура сервиса

Сервис реализован на FastAPI с ASGI-сервером `uvicorn`. Архитектура разделена на четыре слоя:

- `service/core/` — конфигурация (`pydantic-settings`, чтение из переменных окружения и файла `.env`) и обработка ошибок.
- `service/services/` — бизнес-логика: singleton `AgePredictor`, прогрев при старте, вынос блокирующего инференса в пул потоков (`asyncio.to_thread`) с ограничением параллельности через `asyncio.Semaphore(MAX_CONCURRENCY)`.
- `service/api/` — маршруты, `pydantic`-схемы ответов, валидация загрузок.
- `web/static/` — браузерный интерфейс, смонтированный как `StaticFiles` на `/`.

Модель загружается однократно при старте через механизм `lifespan` FastAPI и переиспользуется для всех запросов. Там же выполняется прогрев (`warmup`) — один фиктивный прямой проход, исключающий задержку первого реального запроса.

Старт сервиса (`service/main.py`):

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_predictor()
    yield

app = FastAPI(title="Age Prediction Service", lifespan=lifespan)
app.add_exception_handler(ServiceError, service_error_handler)
app.mount("/", StaticFiles(directory=static_dir, html=True))
```

5 API

Сервис предоставляет пять REST-эндпоинтов с JSON-ответами. Изображения передаются как `multipart/form-data`. Документация OpenAPI (Swagger UI) генерируется автоматически и доступна по адресу `/docs`.

Метод	Путь	Назначение
GET	<code>/health</code>	Liveness-проба — сервис жив
GET	<code>/ready</code>	Readiness — 200 если модель загружена, иначе 503
GET	<code>/v1/meta</code>	Версия модели, лимиты, статистика кэша
POST	<code>/v1/predict</code>	Предсказание возраста по одному изображению
POST	<code>/v1/predict_batch</code>	Пакетное предсказание (до MAX_BATCH_SIZE изображений)

Таблица 2: Конечные точки API

Пример запроса и ответа:

```
curl -F file=@face.jpg http://127.0.0.1:8000/v1/predict

{
  "age": 34.20,
  "age_bin": "30-39",
  "cached": false,
  "model_version": "ab12cd34"
}
```

Поле `model_version` — первые 8 символов SHA-256 файла весов. Поле `age_bin` — возрастной класс по разбивке UTKFace (0–2, 3–12, 13–19, 20–29, 30–39, 40–49, 50–59, 60–69, 70–79, 80+). Поле `cached` сигнализирует об ответе из кэша без прямого прохода модели.

6 Кэширование вычислений

Прямой проход бэкбона — самая дорогая операция (≈ 7 мс на CPU). В сервисе применён двухслойный LRU-кэш, унаследованный из ГП-5. Порядок поиска:

- **Слой 1 — PredictionCache.** Ключ: SHA-256 байт изображения $\rightarrow$ float age. Попадание: ответ без единого обращения к модели.
- **Слой 2 — EmbeddingCache.** Ключ: SHA-256 $\rightarrow$ 512-мерный эмбеддинг. Попадание: пропускается тяжёлый backbone-прогон; запускается только лёгкая FC-голова (Dropout $\rightarrow$ Linear(512, 1)).
- **Холодный промах.** Полный прямой проход; результаты сохраняются в оба кэша (LRU, ёмкость CACHE_SIZE=10 000 по умолчанию).

Статистика кэша (hits, misses, hit_rate для каждого слоя) доступна в реальном времени через GET `/v1/meta` $\rightarrow$ поле `cache_stats` и отображается на вкладке «Состояние системы» браузерного интерфейса.

Сценарий	Время, 50 изобр.	мс / изображение	Ускорение
Холодный проход (нет кэша)	1 002 мс	20.1 мс	1×
Тёплый проход (PredictionCache)	620 мс	12.4 мс	1.6×
EmbeddingCache (backbone skip)	633 мс	12.7 мс	1.6×

Таблица 3: Бенчмарк кэша — 50 изображений, CPU (из ГП-5)

7 Быстродействие

Помимо кэширования, применены следующие меры для снижения задержки:

- `torch.inference_mode()` — все прямые проходы выполняются в режиме инференса (быстрее `torch.no_grad()`, полностью отключает autograd).
- `asyncio.to_thread` + `Semaphore`. Блокирующий CPU-инференс вынесен в пул потоков; event loop не блокируется. Семафор `cap = MAX_CONCURRENCY=4` исключает перегрузку при всплеске запросов.
- Warmup при старте. Один фиктивный прямой проход в `lifespan` — JIT-компиляция и прогрев BLAS-процедур до первого реального запроса.

Модель	Размер	Latency	Throughput
Distilled ResNet18 (float32)	42.7 МБ	7.0 мс	143 img/s
Dynamic quantized	42.7 МБ	7.2 мс	138 img/s
PTQ int8	10.7 МБ	9.8 мс	102 img/s
Pruned + fine-tuned	42.7 МБ	6.9 мс	146 img/s

Таблица 4: Задержка инференса на CPU, single-image (из ГП-5)

8 Надёжность

- **Health/readiness-пробы.** `GET /health` — liveness (сервис жив). `GET /ready` — readiness: возвращает 503, пока модель не загружена; 200 + `model_version` после успешного lifespan. Совместимо с HEALTHCHECK Docker.
- **Валидация загрузок.** Проверка Content-Type (JPEG/PNG/WebP/BMP); ограничение размера `MAX_UPLOAD_BYTES` (по умолчанию 5 МБ → 413); отклонение пустых файлов → 400; битых изображений → 422.
- **Структурированные ошибки.** Все исключения сервиса возвращают JSON `{message, code, status}` с корректным HTTP-статусом через единый FastAPI exception handler (`ServiceError` и его подклассы).
- **Ограничение параллельности.** `asyncio.Semaphore(MAX_CONCURRENCY)` защищает CPU от одновременного запуска более N прямых проходов.

9 Веб-интерфейс

Браузерный интерфейс (`web/static/`) реализован на чистом HTML/CSS/JavaScript без внешних фреймворков и обслуживается самим сервисом как статика (`StaticFiles mount` на `/`). Интерфейс содержит три вкладки:

- «Одно изображение» — drag-and-drop или выбор файла, превью изображения, кнопка «Предсказать», отображение возраста, возрастного бина и бейджа «из кэша» при повторной загрузке того же фото.
- «Пакет» — загрузка нескольких изображений, grid-отображение результатов с предсказанным возрастом для каждого.
- «Состояние системы» — вызов `GET /v1/meta`, отображение backbone, версии модели, лимитов и live-статистики кэша (`hits / misses / hit_rate` для обоих слоёв).

10 Инфраструктура развёртывания

Для локального запуска достаточно Python-окружения с установленными зависимостями:

```
pip install fastapi "uvicorn[standard]" pydantic-settings python-multipart
uvicorn service.main:app --reload

open http://127.0.0.1:8000

open http://127.0.0.1:8000/docs
```

Для контейнеризации подготовлен Dockerfile: базовый образ `python:3.11-slim`, CPU-версия PyTorch (без CUDA), копирование `src/`, `service/`, `web/` и чекпоинта.

```
docker build -t age-app .
docker run -p 8000:8000 age-app

docker run -p 8000:8000 -e MODEL_PATH=/data/model.pth age-app
```

11 Итог

Реализован работающий веб-сервис предсказания возраста с браузерным интерфейсом и REST API, переиспользующий дистиллированный бэкбон ResNet18 (ГП-5) через общий слой инференса `src/inference/`.

Критерий	Решение
Работающий сервис	9/9 pytest-тестов; <code>uvicorn service.main:app</code> запускается, все эндпоинты отвечают
Веб-приложение	Браузерный UI (HTML/CSS/JS), 3 вкладки, drag-and-drop, отображение результатов
Качество кода	Аннотации типов, разделение на слои, pytest TestClient, README, <code>.env.example</code>
Быстродействие	Однократная загрузка + warmup; <code>torch.inference_mode()</code> ; двухслойный LRU-кэш; <code>asyncio.to_thread</code> + <code>Semaphore</code>
Надёжность	Health/readiness (503 до загрузки); валидация загрузок (400/413/422); структурированные ошибки JSON

Таблица 5: Соответствие реализации критериям задания